

E-commerce Microservices Software Design Document

Low-scale backend MVP | 8-week plan | Developer 1, Developer 2, Developer 3

Document	Value	Project	Value
Version	2.0	Prepared	2026-09-09
Audience	Manager and development team	Delivery	Backend-only MVP

1. Overview & Goals

What are we building and why?

We are building a small e-commerce backend using three microservices: User & Authentication, Product & Inventory, and Order. The goal is to deliver a complete, understandable system that teaches REST APIs, security, databases, service-to-service communication, testing, Docker, and teamwork. The first release stays intentionally small, while service and database boundaries support future growth.

Who is it for?

User	Need	MVP access
Customer	Create an account, browse products, place and manage orders	Public and authenticated REST APIs
Administrator	Create/update products and adjust stock	ADMIN-protected REST APIs
Development team	Learn and demonstrate a complete microservices workflow	Local Docker environment, tests, API collection

Core features that mean the MVP is done

#	Core feature	Completion result
1	Customer registration and login	Passwords are hashed and login returns a valid JWT.
2	Catalog and stock administration	Customers can browse active products; admins can create/update products and adjust stock.
3	Order creation	Order Service reserves stock through Product Service, uses server-owned prices, and stores a CONFIRMED order.
4	Order history and cancellation	Customers can view only their orders and cancel an eligible order; reserved stock is released once.
5	Repeatable local delivery	A clean clone starts with Docker Compose; database migrations and automated tests succeed.

MVP boundary

Included now: backend APIs, three services, three PostgreSQL databases, JWT security, Docker Compose, tests, and API documentation. Deferred: frontend, Payment Service, Notification Service, shipping, Kafka, Redis, Kubernetes, and AWS. Payment is excluded initially because money handling, gateways, retries, refunds, and security rules would make the beginner MVP unnecessarily risky.

2. User Flow & Design

The current product is backend-only, so API use cases replace screens for the MVP.

Backend user flows now; the same steps become screens when a frontend is added

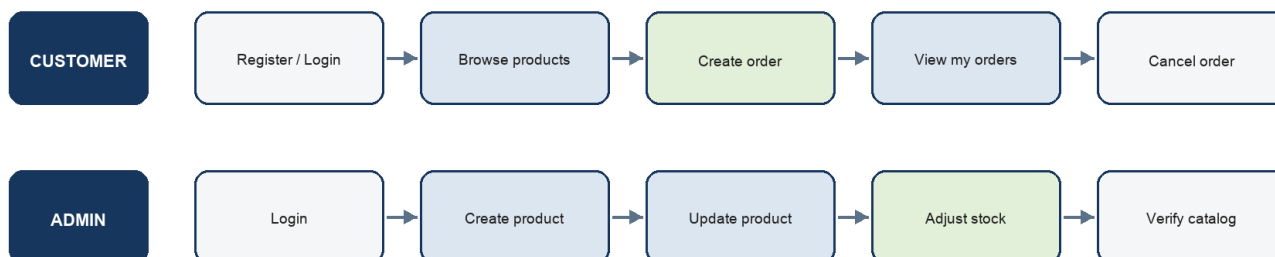


Figure 1. Primary customer and administrator journeys.

Quick screen/page list for the future frontend

Future screen	What the user does	Backend API group used now
Register / Login	Create an account and receive a session token	POST /api/v1/auth/register; POST /api/v1/auth/login
Product Catalog	Browse active products and view details	GET /api/v1/products; GET /api/v1/products/{id}
Admin Products	Create or edit a product	POST /api/v1/admin/products; PUT /api/v1/admin/products/{id}
Admin Stock	Increase, decrease, or set available stock	PATCH /api/v1/admin/products/{id}/stock
Checkout	Choose quantities and submit an order	POST /api/v1/orders
My Orders	List orders and open order details	GET /api/v1/orders; GET /api/v1/orders/{id}
Cancel Order	Cancel a CONFIRMED order	POST /api/v1/orders/{id}/cancel

End-to-end flow and expected behavior

Flow	Service sequence	Success	Failure handling
Login	Client -> User Service	JWT contains user ID and role	Invalid credentials return 401 without revealing which field was wrong
Create order	Client -> Order -> Product	Stock reserved; price snapshot stored; order CONFIRMED	Invalid/insufficient stock returns a clear error and no order is saved
View order	Client -> Order	Only the order owner can read it	Missing order returns 404; another customer receives 403
Cancel order	Client -> Order -> Product	Stock released once; status CANCELLED	Repeated cancellation returns the same final state; partial failure is retried or compensated
Admin update	Client -> Product	Product or stock change is recorded	Non-admin token returns 403; invalid quantity returns 400

Shared design rules

The JWT identifies the caller; clients never choose a customer ID. Product Service owns product prices and stock. Order Service stores a price snapshot and never trusts a total sent by the client. Each protected request validates identity and role, and every error uses a consistent JSON structure with timestamp, status, code, message, path, and correlationId.

3. Data & Architecture

Simple ownership rules prevent accidental coupling and make later scaling possible.

Low-scale MVP architecture with independent service databases

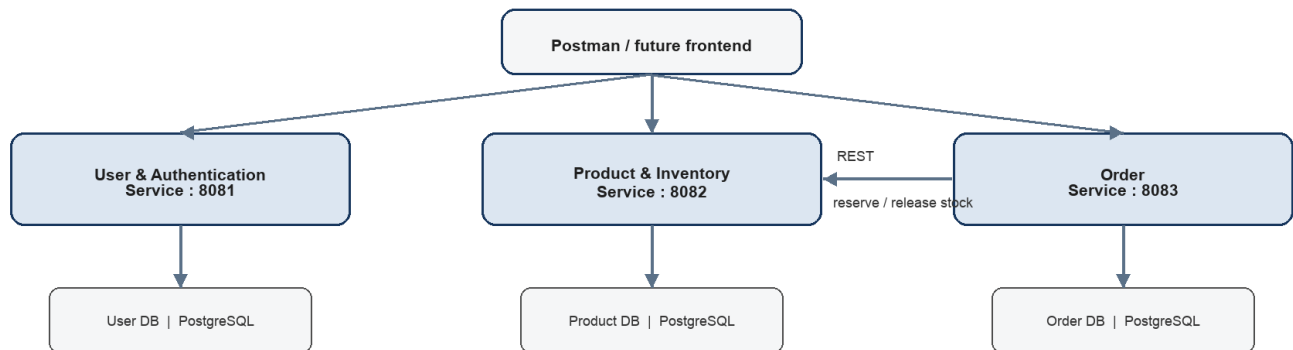


Figure 2. Three services, three databases, and one synchronous service call.

Data ownership and table layout

Service / database	Main tables	Important data and rule
User & Authentication / user_db	users, addresses	users: id, email, password_hash, role, status, timestamps. Email is unique; only this service stores passwords.
Product & Inventory / product_db	categories, products, inventory, reservations, reservation_items, stock_adjustments	Product details, current price, available/reserved stock, reservation key and audit history. Stock cannot become negative.
Order / order_db	orders, order_items, order_status_history	Customer ID, status, totals, product/price snapshots, timestamps. Orders never join another service database.

Key APIs

Service	Public / admin APIs	Internal API
User & Authentication	POST /auth/register, POST /auth/login, GET/PUT /users/me, address CRUD	None in MVP; JWT is verified by each service
Product & Inventory	GET /products; admin product create/update and stock adjustment	POST /internal/inventory/reserve; POST /internal/inventory/release
Order	POST /orders; GET /orders; GET /orders/{id}; POST /orders/{id}/cancel	Calls Product Service reserve/release endpoints

Architecture and external tools

Area	MVP decision
Communication	JSON over REST. Order -> Product uses short timeouts, a correlation ID, and an internal API key.
Security	Spring Security with signed JWTs. Roles: CUSTOMER and ADMIN. BCrypt password hashing. Secrets come from environment variables.
Persistence	One PostgreSQL database per service. Flyway owns schema changes. No service reads another service database.
Developer tools	Docker Compose, GitHub, Maven, Postman/OpenAPI, JUnit, Mockito, Testcontainers, and WireMock.
Operations	Spring Boot Actuator health endpoint and structured application logs. No external payment or cloud dependency in the MVP.

4. Implementation Plan

Build vertical slices early, then integrate and harden the system.

Temporary service ownership

Owner	Primary responsibility	Collaboration expectation
Developer 1	User & Authentication Service	Publish JWT contract and test tokens
Developer 2	Product & Inventory Service	Publish product and reservation contracts
Developer 3	Order Service	Integrate identity, pricing, inventory, and cancellation

The team may swap assignments before implementation starts; this document keeps neutral Developer 1/2/3 labels.

Eight-week build plan

Week	Team deliverable	Exit check
1	Create the monorepo, three Spring Boot skeletons, shared API/error rules, Docker databases, health checks, and branch protection.	Services start; README covers setup.
2	Create data models, Flyway migrations, repositories, validation, sample data, and OpenAPI drafts.	Each service initializes only its database.
3	Build registration, login, BCrypt, JWT validation, roles, profile, and address APIs.	Customer/admin token tests pass.
4	Build catalog, admin product, and stock-adjustment APIs with authorization and audit data.	Admin manages products; customer browses.
5	Build atomic stock reserve/release and connect Order to Product with timeouts and error mapping.	Concurrency tests prevent overselling.
6	Build create/list/get/cancel order flows, ownership checks, price snapshots, history, idempotency, and compensation.	Happy and failure paths pass end to end.
7	Run cross-service tests; improve logs, correlation IDs, security, Docker Compose, and clean-clone setup.	Tests and one-command startup succeed.
8	Fix defects, finalize OpenAPI/Postman and README, rehearse the demo, and obtain manager review.	Acceptance demo and documentation complete.

Technology stack

Layer	MVP choice	Later, when needed
Application	Java 21, Spring Boot 3.x, Maven	API Gateway and centralized configuration
Data and security	PostgreSQL, Flyway, Spring Security, JWT	Redis caching and stronger key management
Integration and testing	REST, OpenAPI/Postman, JUnit, Mockito, Testcontainers, WireMock	Kafka events and broader performance testing
Delivery	Docker Compose and GitHub monorepo	GitHub Actions/Jenkins, Kubernetes, and AWS

GitHub workflow, definition of done, and growth path

Workflow: create a small feature branch -> commit a focused change -> open a pull request -> obtain one teammate review -> merge into protected main. No direct feature pushes to main. A feature is done when its contract, migration, validation, security, tests, error cases, and README note are complete. Manager acceptance requires a clean-clone startup, successful customer/admin demo, passing tests, and a team walkthrough. Next growth steps are Notification Service, Kafka, Redis, Payment Service, CI/CD, Kubernetes/AWS, and a frontend when product needs justify them.